

Lab 4 — VibeCoding Chat History

Chat history used for the Snake Game implementation and Git workflow.

You are working inside my cloned GitHub repository for the Lab 4 VibeCoding assignment.

Your first task is ONLY to inspect and understand the repository. Do NOT modify any files yet.

Please:

1. Inspect the complete repository structure. 2. Read the README.md completely. 3. Identify: - The application's purpose. - The Python entry-point file. - All source files. - Any assets/resources. - requirements.txt or other dependency files. - All features requested in the README. - Any known broken functionality or TODOs. 4. Understand how the current game works. 5. Determine how the project should be executed. 6. Identify every change required to satisfy the README. 7. Create a clear implementation checklist ordered from: - broken-code fixes first - then README features one by one - then testing/cleanup. 8. Do not implement anything yet.

At the end, show me:

- Repository structure - Current execution command - Existing functionality - Problems/bugs found - Every required feature from README - Proposed implementation order

Wait for my next instruction before modifying files.

1:22 AM

Worked for 50s

Repository structure

```
45_snake/ ■■■ main.py ■■■ README.md ■■■ requirements.txt ■■■ game/ ■■■ game_engine.py ■■■ snake.py
■■■ food.py ■■■ __pycache__ / # Python bytecode, not source assets
```

There are no sound, image, or other game resource files. `requirements.txt` contains only `pygame`. The repository also has a `.git` directory, which I excluded from the project structure above.

Current execution command

From the repository root:

```
pip install -r requirements.txt python main.py
```

`main.py` initializes Pygame, opens a 600×600 window, and runs the game loop at 60 frames per second. The engine moves the snake every 7–8 frames, for a speed of about 8 grid moves per second.

Application purpose and existing functionality

This is a terminal-based Snake game project implemented with Pygame graphics. The current code provides:

- Grid-based movement with arrow keys or WASD. - A snake that grows by one segment when it eats food. - Food spawning that checks the current snake body. - A score display that increments when food is eaten. - Wall and self-collision detection that ends the game.

Problems and incomplete behavior

- **Reversing direction can cause an unfair collision.** `Snake.set_direction()` accepts a direct reversal. The next move can put the head into the snake's body. - **No Game Over screen or input prompt.** The engine prints the final score to the console once and then leaves the game frozen in its last frame. - **No replay or difficulty selection.** The game has a fixed speed and no restart menu. - **No sound effects.** There are no sound assets or sound playback calls. - **Food spawning can loop indefinitely if the board fills.** `Food.respawn()` retries random cells until it finds an unoccupied one; it has no full-board case. - `GameEngine.handle_input()` is currently a no-op ('pass'). - The README's expected behavior says movement should be smooth and grid-based; the code is grid-based, but its direction changes currently apply immediately on keypress.

The working tree has three untracked `game/__pycache__` bytecode files. I did not modify any files.

README requirements

The README asks for:

1. Fair direction changes and more accurate collision handling, including wall and self-collisions. 2. A Game Over screen showing the final score and waiting gracefully for input. 3. A replay option after Game Over with Easy, Medium, and Hard speed choices, or an exit option. 4. Sound effects for eating food and for the Game Over moment.

It also specifies expected behavior: arrow-key or WASD controls, one-segment growth per food, food that does not spawn on the snake, a score that updates as food is eaten, and game end/restart behavior on wall or self-collision.

The submission checklist asks for a 10-second video before changes, a 10-second video after changes, and the complete chat history link. Those are deliverables rather than game code features.

Proposed implementation order

1. Fix direction reversal and collision edge cases, including checking whether moving into the tail as it vacates is handled correctly. 2. Add a Game Over screen with the final score and input handling to exit or continue. 3. Add replay initialization and Easy, Medium, and Hard speed selection. 4. Add food-eaten and Game Over sound effects, including any needed resources or dependency handling. 5. Check the implemented behavior against the README, then clean up stale comments and unused code. 6. Capture the before/after gameplay videos and provide the complete chat history link for submission.

This is an inspection-only report; I have not implemented changes and will wait for your next instruction.

1:23 AM

Now implement ONLY the broken-code fixes you identified from the repository and README.

Requirements:

1. Fix the existing broken functionality without unnecessarily rewriting the project. 2. Preserve the existing architecture and coding style wherever possible. 3. Do not implement the additional README features yet. 4. Make the smallest reasonable changes required to make the existing game work correctly. 5. Handle obvious runtime errors, crashes, incorrect game logic, input problems, and broken functionality. 6. Do not introduce unnecessary dependencies. 7. After making the changes, run the appropriate tests or execute the game to verify the fix. 8. Check that the application starts successfully. 9. Review the modified files for syntax errors and obvious regressions.

When finished, report:

- What was broken - What files were changed - What was fixed - How you tested it - Any remaining issues

Do not implement the next feature yet.

1:23 AM

Worked for 6m 18s

What was broken

- Pressing the opposite direction immediately reversed the snake into its body, causing an unfair collision. - Food spawning could loop forever if no unoccupied grid cells remained.

Files changed

- [game/snake.py](C:/Users/Lenovo/45_snake/game/snake.py): ignores direct reversal input. - [game/food.py](C:/Users/Lenovo/45_snake/game/food.py): selects from available cells and reports when none remain. - [game/game_engine.py](C:/Users/Lenovo/45_snake/game/game_engine.py): ends play if the board is full and food cannot respawn.

Verification

- Ran the app's startup and shutdown with Pygame's dummy display: passed. - Checked reverse and perpendicular direction inputs, movement, and food respawn including a full board: passed. - Ran `python -m compileall -q main.py game` and `git diff --check`: passed.

The Game Over screen, replay/difficulty selection, and sound effects remain unimplemented as requested. The repository still contains untracked bytecode files under `game/\_\_pycache\_\_`; they were already present before these changes.

 Edited 3 files+16-8View changes

game/food.pygame/food.py+11-5

game/game_engine.pygame/game_engine.py+2-1

game/snake.pygame/snake.py+3-2

1:30 AM

Implement ONLY the FIRST feature listed in the README.

Before coding:

1. Restate the exact feature requirement from the README. 2. Identify which files need to change. 3. Explain briefly how you will implement it.

Then implement it.

Requirements:

- Follow the README requirement exactly. - Preserve existing functionality. - Do not implement other README features yet. - Keep the implementation simple and maintainable. - Do not unnecessarily rewrite working code. - Do not add dependencies unless absolutely necessary. - Make the feature work correctly in the actual running game.

After implementation:

1. Run the application. 2. Test the new feature. 3. Verify that existing functionality still works. 4. Fix any errors caused by the implementation. 5. Review the final diff.

Finally report:

- Feature implemented - Files changed - How it works - How it was tested - Any remaining limitations

Do not implement the next feature.

1:30 AM

Worked for 34s

Feature requirement

The README's first feature is **Refine Collision Detection**: "The snake can die instantly and unfairly if you tap the opposite direction key while moving (e.g. pressing Left while already moving Right), because the new head moves straight into the segment behind it. Investigate and enhance collision accuracy so direction changes and self/wall collisions behave fairly."

Implemented

In [game/snake.py](C:/Users/Lenovo/45_snake/game/snake.py), direct reversals are ignored, and the snake can make at most one turn between grid moves. The turn lock clears after movement, so a new turn is accepted on the next step. Existing wall and self-collision checks remain in place.

Verification

- Started and exited `main.py` with Pygame's dummy display: passed. - Checked direct and rapid reverse inputs, valid turns, wall and self-collision detection, food scoring, and snake growth: passed. - Checked Python syntax and `git diff --check`: passed.

Files and remaining limitations

This feature changed `game/snake.py`. The working tree also retains earlier broken-code fixes in `game/food.py` and `game/game\_engine.py`. The Game Over screen, replay/difficulty selection, and sound effects remain unimplemented. The startup check used a dummy display, so I did not verify the game interactively in a visible window.

 Edited snake.py+8-1View changes

1:30 AM

The first README feature is now implemented and tested.

Before committing:

1. Run `git status`. 2. Run `git diff` and review all changes. 3. Make sure only changes related to Feature 1 and its necessary fixes are included. 4. Do not include unrelated modifications. 5. Do not modify files just to create a commit.

Then create a clear Git commit:

```
git add \<only the relevant files>\ git commit -m "Add \<feature name>"
```

After committing, run:

```
git status\ git log -1 --oneline
```

Show me the commit hash and commit message.

Do not push yet.

1:31 AM

Worked for 19s

Committed the reviewed game source changes locally. The untracked `game/\_\_pycache\_\_` files were excluded.

- **Commit:** `330832f` - **Message:** `Add fair collision detection` - **Status:** Branch is one commit ahead of `origin/main`; no tracked changes remain. - **Push:** Not pushed.

1:31 AM

Now implement ONLY the SECOND feature listed in the README.

Do not modify or redesign unrelated parts of the project.

First:

1. Identify the exact second feature requirement from README.md. 2. Explain the implementation approach. 3. Identify the files that need modification.

Then: 4. Implement the feature. 5. Run the game. 6. Test the feature thoroughly. 7. Verify that Feature 1 still works. 8. Fix any regression or runtime error. 9. Review the git diff.

Do not implement any later features.

At the end, report:

- Exact requirement - Implementation - Files changed - Testing performed - Result - Any limitations

1:32 AM

Worked for 25s

Exact requirement

The README's second feature, **Implement Game Over Condition**, says: "Add a screen that displays the final score once the snake hits a wall or itself, then gracefully waits for input instead of just printing to the console."

Implementation

The game now displays a Game Over overlay with the final score and an exit prompt. The game remains frozen until the player presses a key or closes the window. Pressing a key exits; replay behavior was not added.

Files changed

- [game/game_engine.py](C:/Users/Lenovo/45_snake/game/game_engine.py): renders the Game Over screen and handles key input while the game is over. - [main.py](C:/Users/Lenovo/45_snake/main.py): exits the main loop when that input is received.

Testing and result

- Ran the app with Pygame's dummy display and confirmed clean startup and exit. - Triggered a wall collision, checked that the game-over state froze, rendered the screen, and verified that key input requests exit. - Rechecked Feature 1's direct and rapid reversal handling. - Checked syntax and `git diff --check`: passed.

The feature works in the tested Pygame environment. I used a dummy display, so I did not verify the screen interactively in a visible window.

 Edited 2 files+24-5View changes

game/game_engine.pygame/game_engine.py+22-5

main.pymain.py+2-0

1:32 AM

Feature 2 has been implemented and tested.\ Review the changes using:\ git status\ git diff\ Ensure only Feature 2 changes and necessary supporting changes are included.\ Then create a separate commit:\ git add \<relevant files>\ git commit -m "Add \<feature name>"\ Then verify:\ git status\ git log --oneline -3\ Do not push yet.

1:32 AM

Worked for 32s

Committed Feature 2 separately.

- **Commit:** `009a920 Add game over screen` - **Files committed:** `game/game\_engine.py`, `main.py` - **Status:** Two commits ahead of `origin/main`; no tracked changes remain. Untracked `game/\_\_pycache\_\_` files were left out. - **Push:** Not pushed.

1:33 AM